

Mongoose Core Methods — Interview Notes

1. Create Operations

Model.create(data)

```
await Task.create({ title: "Task 1" });
```

Use when:

- Simple insert
- Minimal preprocessing

Behavior:

- Runs validation
- Saves immediately
- Returns created document

new Model() + save()

```
const task = new Task({ title: "Task 1" });
```

```
await task.save();
```

Use when:

- You need to modify data before saving
- You want full control (pre-save logic, hooks)

2. Read Operations

findById(id)

```
await Task.findById(id);
```

Use when:

- Fetching by `_id`

findOne(filter)

```
await Task.findOne({ title: "task 1" });
```

Use when:

- Expecting a single document

find(filter)

```
await Task.find({ status: "todo" });
```

Use when:

- Fetching multiple documents

exists(filter)

```
await Task.exists({ title: "task 1" });
```

Use when:

- Only checking if a document exists (faster than find)

countDocuments(filter)

```
await Task.countDocuments({ status: "todo" });
```

Use when:

- Counting records (pagination use case)

3. Update Operations**findByIdAndUpdate(id, update, options)**

```
await Task.findByIdAndUpdate(id, updateData, {  
  new: true,  
  runValidators: true  
});
```

Important options:

- new: true → returns updated document
- runValidators: true → enables validation during update

updateOne(filter, update)

```
await Task.updateOne({ _id: id }, { status: "done" });
```

Use when:

- Updating without returning the document

updateMany(filter, update)

```
await Task.updateMany(  
  { status: "todo" },  
  { status: "in-progress" }  
);
```

Use when:

- Bulk updates

4. Delete Operations**findByIdAndDelete(id)**

```
await Task.findByIdAndDelete(id);
```

deleteOne(filter)

```
await Task.deleteOne({ _id: id });
```

deleteMany(filter)

```
await Task.deleteMany({ status: "done" });
```

5. Populate (Relationships)

Basic populate

```
await Task.find().populate("assignedTo", "name email");
```

Purpose:

- Replace ObjectId with actual referenced document

Nested populate

```
await Task.find().populate({  
  path: "project",  
  populate: {  
    path: "owner",  
    select: "name"  
  }  
});
```

Use when:

- Multi-level relationships

6. Query Helpers (Chaining)

Select fields

```
await Task.find().select("title status");
```

Sort

```
await Task.find().sort({ createdAt: -1 });
```

- 1 → ascending
- -1 → descending

Limit

```
await Task.find().limit(10);
```

Skip (Pagination)

```
await Task.find().skip(10);
```

7. Query Operators (Important)

Comparison

```
{ dueDate: { $lt: new Date() } }
```

Not equal

```
{ status: { $ne: "done" } }
```

OR condition

```
{ $or: [{ status: "todo" }, { status: "in-progress" } ] }
```

IN operator

```
{ status: { $in: ["todo", "done"] } }
```

Regex search

```
{ title: { $regex: "fix", $options: "i" } }
```

8. Indexing

Example (compound unique index)

```
taskSchema.index(  
  { title: 1, assignedTo: 1 },  
  { unique: true }  
);
```

Purpose:

- Prevent duplicates
- Improve query performance

9. Validation Notes

- create() and save() → validators run automatically
- findByIdAndUpdate() → validators do NOT run by default

Enable explicitly:

```
{ runValidators: true }
```

10. Essential Methods (Shortlist)

Students must be confident in:

- create
- find
- findOne

- findById
- findByIdAndUpdate
- findByIdAndDelete
- populate
- select
- sort
- limit
- skip

11. Example (Interview-Level Query)

Problem:

Find tasks assigned to a user, not completed, sorted by due date, return only title.

```
await Task.find({  
  assignedTo: userId,  
  status: { $ne: "done" }  
})  
.select("title")  
.sort({ dueDate: 1 });
```